

Experimental Evaluation of EC2-Based and Serverless Architectures for Bursty Web Application Workloads on AWS: Performance, Scalability, and Cost Trade-offs

Preprint

Shaheen Sardar

Independent Researcher, Pakistan

Corresponding author: shaheensardar.it@gmail.com

Abstract

Cloud applications can be deployed using continuously provisioned virtual machines or serverless platforms that allocate computing resources on demand. Choosing between these approaches is not straightforward because performance, scalability, reliability, and cost can change depending on workload characteristics. This study experimentally compares a fixed-capacity Amazon EC2 deployment with an AWS serverless architecture using API Gateway and Lambda. Both implementations executed the same deterministic CPU-bound /compute workload based on 25,000 chained SHA-256 operations. The EC2 environment used a t3.small instance running Ubuntu 24.04 with Flask, Gunicorn, and Nginx, while the serverless environment used AWS Lambda with Python 3.12, x86_64 architecture, and 1024 MB of memory.

Six workload profiles were evaluated: low, moderate, high, sudden burst, idle-to-burst, and sustained load. Each workload was executed three times on both architectures, producing 36 accepted formal trials. Because only three repetitions were available for each condition, the analysis focuses on descriptive statistics rather than statistical-significance claims. The results show that neither architecture was consistently superior across all conditions. During the sudden-burst workload, Serverless reduced mean latency by 32.1% and increased achieved throughput by 14.5%, although it also showed a 0.434% mean failure rate and a normalized cost per 1,000 successful requests 9.50 times that of EC2. In the idle-to-burst workload, Serverless reduced mean latency by 41.9% and increased throughput by 30.3%, while its normalized cost moved much closer to EC2 at 1.06 times the EC2 value. Under sustained load, Serverless achieved slightly higher throughput but showed substantially higher P95 latency and a higher failure rate. Overall, the findings indicate a workload-dependent trade-off: Serverless was often more responsive under burst-oriented traffic, whereas the tested EC2 configuration generally provided stronger request reliability and lower normalized cost. The observed Lambda concurrency ceiling of 10 is treated only as an account-specific experimental constraint and not as a general AWS Lambda scalability limit.

Keywords— Cloud computing, serverless computing, Amazon EC2, AWS Lambda, burst workloads, performance evaluation, scalability, tail latency, cloud cost.

1. Introduction

1.1 Background

Cloud platforms provide several ways to deploy and operate web applications. One of the most common approaches is to use virtual machines, where the user provisions computing capacity and manages much of the operating system and application environment. A different approach is serverless computing, particularly Function-as-a-Service (FaaS), where the cloud provider takes responsibility for much of the underlying resource provisioning, scaling, and runtime management.

This difference in responsibility can make serverless platforms attractive for applications with changing or unpredictable traffic. However, reducing infrastructure-management effort does not necessarily make application behavior simpler. Previous research has shown that commercial FaaS platforms can be affected by execution-environment initialization, isolation mechanisms, resource allocation, and workload intensity [1], [2]. Production traces have also shown that serverless applications may experience large variations in invocation frequency, making intermittent and bursty demand an important workload pattern to evaluate [3].

AWS Lambda is one of the most widely used FaaS services. Lambda executes functions inside isolated execution environments, with AWS using Firecracker microVM technology as part of its isolation architecture [4]. Amazon EC2 follows a different model: users explicitly provision a virtual machine and retain control over its operating system, application stack, and available capacity.

The pricing models also differ. EC2 charges for provisioned instance capacity while the instance is running, whereas AWS Lambda pricing is largely based on request count and execution duration [12], [13]. Because of these differences, choosing between EC2 and Serverless should not be based only on which architecture appears faster. The decision also depends on workload shape, reliability requirements, scaling behavior, and cost.

1.2 Problem

For architecture selection, broad claims such as “serverless scales better” or “virtual machines are cheaper” are not very useful on their own. A system that behaves well under a steady light load may react differently to a sudden burst, and an architecture with low average latency can still have poor tail latency or a higher failure rate. Throughput, reliability, and cost may also move in opposite directions. This makes high-percentile latency important: as Dean and Barroso note, tail delays can strongly affect user-perceived behavior even when averages look acceptable [9].

1.3 Research Gap

Previous work has examined commercial serverless platforms [2], large production FaaS workloads [3], cold-start and launch overheads [5], [6], alternative runtime and isolation designs [7], and performance/cost modeling for FaaS applications [8]. The EC2-versus-serverless comparison is therefore not presented here as a new research problem by itself. The contribution of this study is a small, reproducible case study in which the application computation, endpoint, client behavior, AWS Region, workload definitions, trial count, metric-extraction procedure, and cost method were fixed before the formal analysis. The goal is to show how the trade-offs change across workload patterns, not to produce a universal ranking of the two AWS services.

1.4 Research Questions

This study addresses four research questions:

RQ1. How do EC2 and serverless architectures differ in average latency, P95/P99 tail latency, and achieved throughput across low, moderate, high, burst, idle-to-burst, and sustained workloads?

RQ2. How do reliability and observed scaling behavior change as concurrency and burstiness increase?

RQ3. How does estimated cost per 1,000 successful requests differ between continuously provisioned EC2 capacity and request/duration-based serverless execution?

RQ4. How should resource behavior and an account-specific Lambda concurrency quota be interpreted without generalizing an experimental account constraint to AWS Lambda as a service?

1.5 Contributions

This study makes the following practical contributions:

1. A controlled EC2-versus-serverless experiment using identical deterministic application-level computation.
2. Six frozen workload profiles that distinguish low, moderate, high, sudden-burst, idle-to-burst, and sustained behavior.
3. Thirty-six accepted formal trials, with three repetitions for each workload/architecture condition and descriptive variability reporting.
4. Joint analysis of average latency, median latency, P95/P99 latency, achieved throughput, request failures, CloudWatch resource metrics, and normalized list-price cost.
5. Explicit separation of formal results from pilot calibration data and explicit separation of Lambda platform behavior from the experimental account's concurrency quota.
6. A reproducibility package containing processed data, non-sensitive methodology artifacts, analysis scripts, tables, and figures.

2. Related Work

Hellerstein et al. describe serverless computing as an important direction in cloud systems while also emphasizing the constraints introduced by provider-managed execution [1]. Wang et al. measured AWS Lambda, Azure Functions, and Google Cloud Functions from a customer perspective, including scalability, cold-start latency, isolation, and resource efficiency [2]. Their findings are directly relevant to this work because they show why serverless platforms need to be measured in practice rather than treated as interchangeable abstractions.

Shahrad et al. analyzed production FaaS traces and found large differences in invocation frequency across applications, reinforcing the importance of irregular and bursty demand [3]. Startup behavior is another recurring issue. Boucher et al. reported substantial cold-launch overhead in conventional FaaS paths [6], while Akkus et al. proposed SAND to reduce startup delay and improve elasticity and resource efficiency [5]. AWS also distinguishes cold initialization from reuse of an existing Lambda execution environment [11]. For that reason, the W05 idle-to-burst workload in this study is not labeled a cold-start experiment simply because it includes an idle period.

The underlying isolation mechanism can also influence serverless performance. Agache et al. describe Firecracker, the microVM monitor used for serverless workloads including AWS Lambda [4]. Shillaker and Pietzuch show that alternative isolation approaches can change both throughput and tail latency [7]. These results support measuring more than average response time; the present study therefore includes P95 and P99 latency alongside throughput and failures.

Cost is equally dependent on workload and configuration. Lin et al. developed detailed performance and cost models for FaaS applications and noted the difficulty of optimization when infrastructure behavior is partly opaque [8]. The billing units also differ in real AWS deployments: EC2 On-Demand pricing applies to provisioned instance capacity [13], while Lambda charges for requests and GB-second execution duration [12]. API Gateway adds its own request charge to the serverless HTTP path [15]. To make the comparison easier to interpret, this study reports estimated cost per 1,000 successful requests in addition to estimated per-trial cost.

Dean and Barroso's “tail at scale” analysis explains why high-percentile latency deserves separate attention in interactive systems [9]. The experiment here is much smaller than a warehouse-scale service, but the same measurement principle applies: P95 and P99 can expose behavior that is hidden by an average. For this reason, latency, tail latency, throughput, reliability, and cost are treated as separate outcome dimensions throughout the paper.

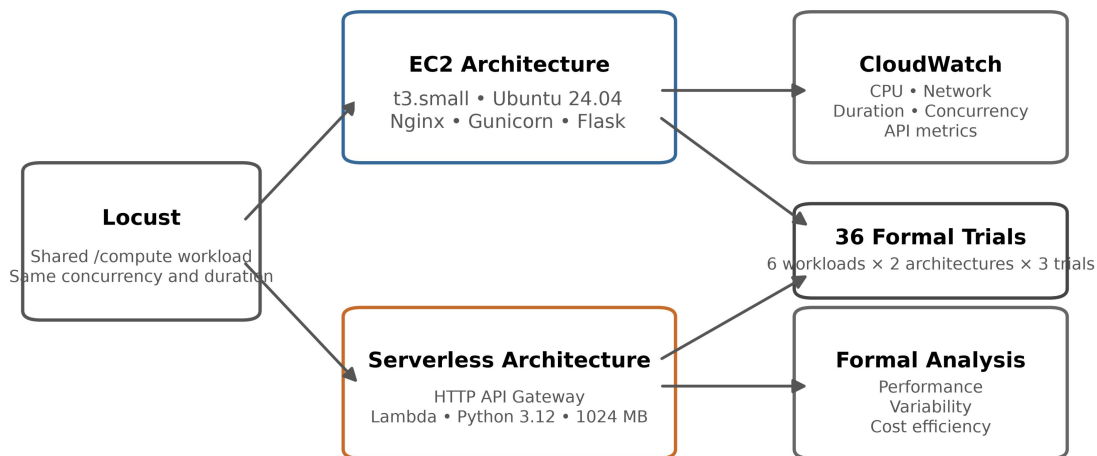
3. Experimental Methodology

3.1 Research Design

The experiment followed a controlled factorial design with repeated trials. The two main factors were architecture (EC2 or Serverless) and workload profile (W01–W06). Measured outcomes included response latency, P95/P99 tail latency, achieved requests per second, request failures, EC2 utilization, Lambda/API Gateway metrics, and estimated cost. To keep the comparison consistent, both architectures used the same AWS Region (ap-south-1), the same /compute behavior, the same Locust client logic, the same workload definitions, and the same analysis pipeline.

The final dataset contains 36 formal trials: six workloads $\times$ two architectures $\times$ three repetitions. With only three observations per condition, the results are summarized using means, sample standard deviations, ranges, architecture ratios, and relative differences. The paper does not report p-values or claim statistical significance from these $n=3$ groups.

Experimental Architecture and Measurement Workflow



Result figures use accepted formal W01-W06 trials only; pilot calibration data are excluded.

Figure 1. Experimental architecture and measurement workflow.

3.2 EC2 Architecture

The EC2 deployment used a Linux t3.small instance in ap-south-1 and was provisioned with Terraform. The Flask application ran in a Python virtual environment. Gunicorn was managed by systemd, and Nginx served as the reverse proxy for the /compute endpoint. The instance had an 8 GB encrypted gp3 EBS root volume and a public IPv4 endpoint. Amazon Systems Manager was enabled for management, and detailed EC2 CloudWatch monitoring supplied the infrastructure telemetry used in the formal analysis.

The choice of t3.small matters when interpreting the results. T3 instances are burstable and rely on CPU credits; AWS documents t3.small as a 2-vCPU instance with a 20% baseline per vCPU, and T3 instances normally launch in Unlimited mode unless the account configuration changes that behavior [14]. The EC2 measurements in this paper therefore describe this specific burstable setup rather than EC2 performance in general.

3.3 Serverless Architecture

The serverless path exposed the application through Amazon API Gateway HTTP API and executed the compute logic in AWS Lambda. The Lambda function used Python 3.12 on x86_64 with 1024 MB of memory, and Terraform managed

the related resources. CloudWatch metrics for API Gateway and Lambda were collected separately. This separation makes it possible to distinguish API-layer latency and errors from function-level execution, concurrency, errors, and throttling.

AWS defines Lambda concurrency as the number of simultaneous in-flight function invocations, and functions can be throttled when the available account concurrency is exhausted [10]. During this experiment, the account exposed a regional concurrency quota of 10. That unusually low quota affected some higher-load runs and is treated throughout the paper as a constraint of the experimental account, not as the normal Lambda service limit.

3.4 Identical Computation

Both architectures executed the same deterministic CPU-bound routine: 25,000 chained SHA-256 operations (sha256-v1). A lightweight /health endpoint was used for health checks, while /compute triggered the measured computation. Before formal testing, EC2 and Lambda responses were checked to confirm that the same input produced the same deterministic result. Features such as authentication, databases, sessions, and file uploads were intentionally left out so that application complexity would not obscure the infrastructure comparison.

3.5 Workload Model

Locust 2.46.3 generated the formal workload. The same Locust file was used for EC2 and Serverless, with only the destination host changed. Each virtual user repeatedly called /compute and waited a randomized 0.2–0.5 s after a completed request before issuing the next one. This is a closed-loop virtual-user workload. As a result, requests per second should be read as achieved throughput under the fixed user-concurrency pattern rather than as an independently fixed offered request rate.

Table 2. Frozen Formal Workload Definitions

Workload	Definition	Concurrent users	Spawn rate (users/s)	Load duration (s)	Pre-load idle (s)
W01	Low	5	1	60	0
W02	Moderate	15	3	60	0
W03	High	30	5	60	0
W04	Sudden-Burst	30	30	60	0
W05	Idle-to-Burst	30	30	60	300
W06	Sustained	20	4	300	0

Note: W05 includes a 300-second idle period before the 60-second burst.

The six profiles were chosen to expose different traffic patterns. W01 represents light demand; W02 increases concurrency gradually; W03 pushes the same pattern further; W04 introduces an abrupt 30-user burst; W05 repeats that burst after a 300-second idle period; and W06 holds 20 users for five minutes. W05 is deliberately described as idle-to-burst rather than as a guaranteed cold-start test because AWS may retain and reuse a Lambda execution environment after an idle interval [11].

3.6 Pilot Calibration

Pilot runs were used before the formal experiment to check instrumentation and to calibrate the workload intensity. These runs were stored separately and were not included in the formal figures or statistical summaries. The original sudden-burst pilot used 50 users spawned at 50 users/s for 60 s. On the serverless path, it produced 405 HTTP 503 failures while Lambda concurrency reached the account quota of 10. That pilot was kept as calibration evidence, and the formal W04 workload was set to 30 users with a 30 users/s spawn rate.

Reducing the formal burst intensity was not intended to remove an unfavorable serverless result. The quota-bound pilot remains part of the preserved evidence and is discussed as an environmental limitation. The adjustment was made so that the six formal workload profiles could be repeated consistently while still recording failures that occurred under the accepted test conditions.

3.7 Formal Experiment Design

Each workload was executed three times on EC2 and three times on Serverless, for a total of 36 formal trials. Trial identifiers used the EC2-Wxx-Tyy and SVL-Wxx-Tyy patterns. The runner recorded UTC start and end times, workload parameters, Locust statistics, failures, exceptions, and the relevant application and Terraform commits. Instrumentation attempts that were invalid were archived separately rather than being silently replaced. A completeness audit later verified all 36 unique trial identifiers and generated a SHA-256 manifest for the retained raw artifacts.

3.8 Metrics

Application-level metrics came from the Locust /compute row and included total, successful, and failed requests; failure rate; average and median response time; P95 and P99 latency; minimum and maximum latency; and requests per second.

For each workload/architecture pair, the three trial values were summarized using the mean and sample standard deviation, with ranges retained to show trial-to-trial spread.

CloudWatch metrics were queried for each formal UTC run window at 60-second granularity. EC2 measurements included CPUUtilization, NetworkIn, NetworkOut, and StatusCheckFailed. Lambda measurements included Invocations, Duration, Errors, Throttles, and ConcurrentExecutions, while API Gateway measurements included Count, 4XX, 5XX, Latency, and IntegrationLatency. Because CloudWatch aggregates timestamps, these values provide infrastructure context for the requested run windows; they should not be interpreted as millisecond-level traces matched to individual Locust requests.

3.9 Cost Methodology

The cost comparison uses a regional AWS list-price snapshot captured during the experiment rather than the researcher's AWS bill. Free Tier usage, credits, Savings Plans, Reserved Instance discounts, taxes, and other account-specific benefits were excluded so that the comparison would not depend on one account's billing arrangement. The dataset therefore records `actual_cost_usd` as `NOT_DIRECTLY_ATTRIBUTABLE` and uses `estimated_cost_usd` for the reproducible list-price estimate.

For EC2, the estimate includes t3.small On-Demand compute, gp3 storage, and the in-use public IPv4 address. Allocation time matches the load duration except for W05, where the 300-second idle interval and the following 60-second burst are both charged to the continuously provisioned EC2 instance. The Serverless estimate uses measured Lambda invocations, average Lambda duration, 1 GB of configured memory, the regional Lambda request and GB-second rates, and the measured API Gateway request count. No Lambda compute cost is assigned during the W05 idle interval. CloudWatch research instrumentation, Systems Manager usage, Git, the load generator, and unrelated account services are outside the cost boundary.

The captured pricing snapshot was:

- EC2 t3.small Linux: 0.0224 USD per Hrs (APS3-BoxUsage:t3.small).
- EBS gp3 storage: 0.0912 USD per GB-Mo (APS3-EBS:VolumeUsage:gp3).
- Lambda standard requests: 2e-07 USD per Request (APS3-Request).
- Lambda standard x86 duration: 1.66667e-05 USD per Lambda-GB-Second (APS3-Lambda-GB-Second).
- API Gateway HTTP API requests: 1.05e-06 USD per Requests (APS3-APIGatewayHttpRequest).
- Public IPv4 in-use: 0.005 USD per Hrs (PublicIPv4:InUseAddress).

Lambda compute cost is therefore an estimate derived from CloudWatch aggregate Duration, invocation count, and configured memory. It is not a reconstruction of every invoice-level billed-duration record, which is why the paper consistently describes these values as estimated cost rather than actual billed cost.

4. Experimental Environment

Table 1. Experimental Environment

Category	Parameter	Value
Cloud region	AWS Region	ap-south-1
Experimental design	Formal trials	36 accepted trials
Experimental design	Conditions	6 workloads × 2 architectures × 3 repetitions
Workload	Compute operation	Chained SHA-256 CPU-bound computation
Workload	Compute iterations	25,000
Load generation	Tool	Locust 2.46.3
EC2	Instance	t3.small
EC2	Application stack	Ubuntu 24.04, Flask, Gunicorn, Nginx
EC2	Storage / access	8 GB gp3 EBS; public IPv4 endpoint
Serverless	Lambda configuration	Python 3.12, x86_64, 1024 MB
Serverless	API	Amazon API Gateway HTTP API
Monitoring	AWS telemetry	Amazon CloudWatch; 60-second formal metric extraction
Cost	Cost methodology	Regional AWS list-price estimate
Cost	Account-specific benefits	Free Tier, credits and discounts excluded

Note: The formal analysis uses the frozen experimental configuration.

The environment was intentionally kept small enough to reproduce and inspect. EC2 and Serverless were deployed in the same AWS Region and exposed equivalent application behavior. The load generator ran outside both target architectures and was checked before formal testing for obvious CPU saturation. Terraform managed the infrastructure definitions, and configuration checks were completed before the formal phase.

5. Results

Only the 36 formal trials are used in the results below; pilot calibration runs are excluded. Error bars in the figures show the sample standard deviation across the three repetitions. Given the small $n=3$ groups, the comparisons are presented as descriptive observations rather than statistically significant population effects.

Table 3. Performance Comparison

Workload	EC2 mean latency (ms)	Serverless mean latency (ms)	Latency Δ	EC2 P95 (ms)	Serverless P95 (ms)	EC2 P99 (ms)	Serverless P99 (ms)	EC2 throughput (req/s)	Serverless throughput (req/s)	Throughput Δ
W01	123.19	126.04	+2.3%	196.67	163.33	390.00	363.33	10.10	10.05	-0.4%
W02	200.09	174.43	-12.8%	730.00	426.67	1326.67	733.33	26.70	27.59	+3.3%
W03	341.52	325.82	-4.6%	720.00	570.00	1380.00	1276.67	43.72	40.00	-8.5%
W04	225.00	152.77	-32.1%	400.00	300.00	563.33	513.33	51.78	59.26	+14.5%
W05	290.49	168.64	-41.9%	396.67	340.00	723.33	676.67	44.04	57.39	+30.3%
W06	572.17	544.83	-4.8%	996.67	1566.67	2373.33	4233.33	20.64	21.84	+5.8%

Note: Values are descriptive means across three accepted formal repetitions. Δ is Serverless relative to EC2.

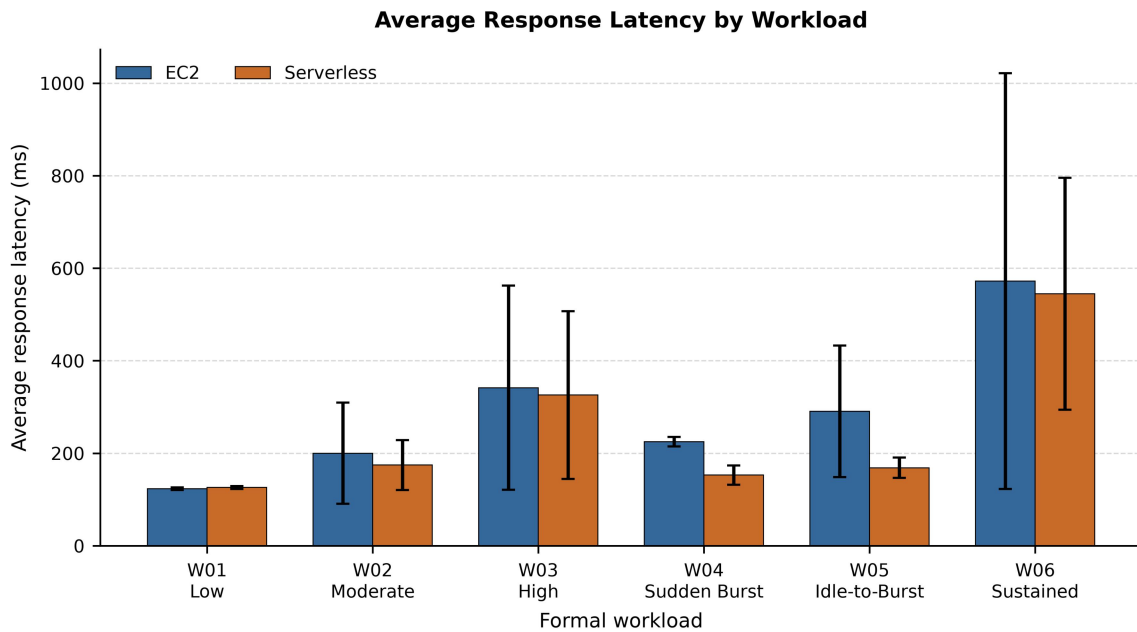


Figure 2. Mean response latency by formal workload; error bars show sample SD across three trials.

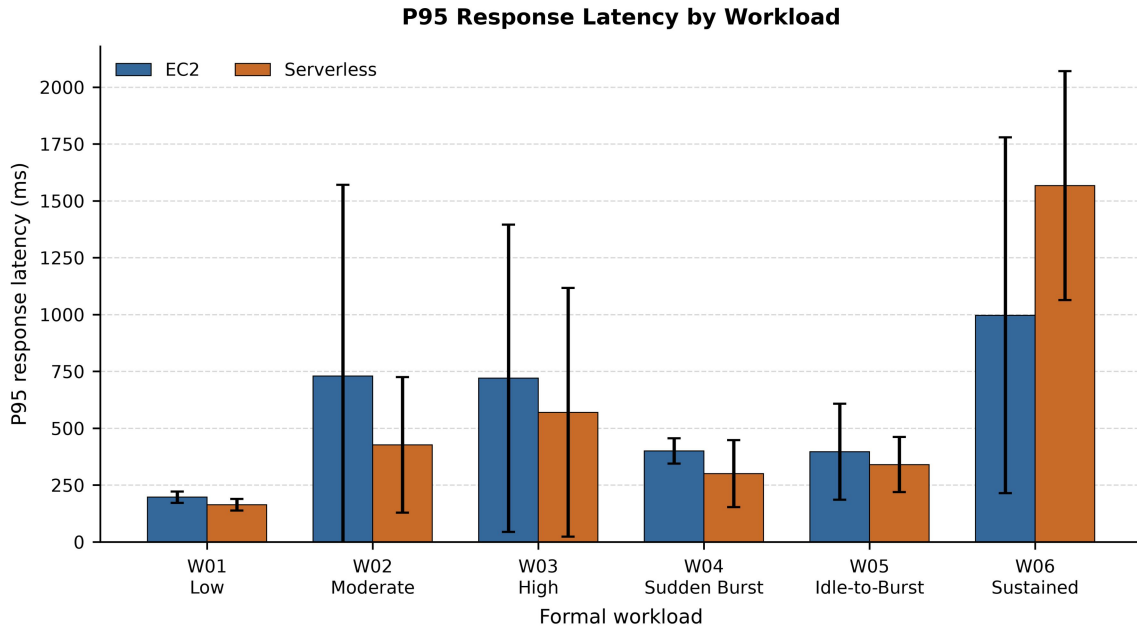


Figure 3. Mean P95 latency by formal workload; error bars show sample SD across three trials.

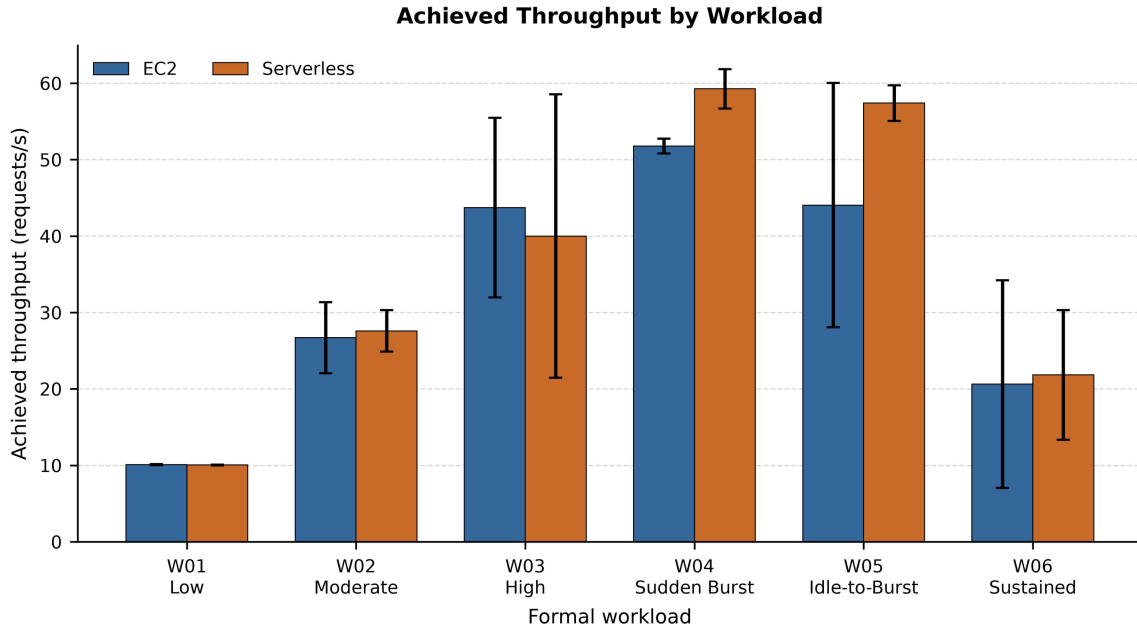


Figure 4. Achieved throughput by formal workload; error bars show sample SD across three trials.

5.1 Low Workload

With five concurrent users, the two architectures performed similarly on the main speed metrics. EC2 recorded 123.19 ± 2.99 ms mean trial-level average latency, compared with 126.04 ± 3.18 ms for Serverless, a 2.3% increase for Serverless. Tail latency slightly favored Serverless: mean P95 was 196.67 ms on EC2 and 163.33 ms on Serverless (-16.9%), while mean P99 was 390.00 ms and 363.33 ms (-6.8%), respectively. Throughput was almost unchanged at 10.10 requests/s for EC2 and 10.05 requests/s for Serverless (-0.4%). Neither architecture recorded request failures in this workload. The larger difference was cost: estimated cost per 1,000 successful requests was \$0.000804 for EC2 and \$0.001485 for Serverless, making Serverless 1.85 $\times$ as expensive in this condition. Overall, W01 did not show a clear performance winner.

At this light load, both deployments had enough capacity for the five-user workload, so the small latency and throughput differences are less important than they become at higher load. In this particular condition, cost was the clearest practical distinction.

5.2 Moderate Workload

At moderate load, Serverless showed lower latency on most of the reported measures. Mean trial-level average latency was 200.09 ± 109.50 ms on EC2 and 174.43 ± 53.99 ms on Serverless, a 12.8% reduction. Mean P95 fell from 730.00 ms

to 426.67 ms (-41.6%), and mean P99 from 1326.67 ms to 733.33 ms (-44.7%). Achieved throughput was 26.70 requests/s for EC2 and 27.59 requests/s for Serverless (+3.3%). Reliability, however, moved in the opposite direction: EC2 had a 0.000% mean failure rate, while Serverless recorded 0.115%, a +0.115 percentage-point difference. Estimated cost per 1,000 successful requests was \$0.000311 for EC2 and \$0.001291 for Serverless, giving a 4.16× Serverless-to-EC2 cost ratio. In W02, the latency advantage of Serverless therefore came with higher cost and a small reliability penalty.

The spread between repetitions is also noticeable in W02, especially for latency. That variability is a reminder that the architecture-level means should not be read as if they came from a large sample; a single run would have given a much less stable picture.

5.3 High Workload

Under the higher W03 load, the comparison became less straightforward. Serverless had slightly lower mean latency: 325.82 ± 181.16 ms versus 341.52 ± 220.69 ms on EC2 (-4.6%). Mean P95 was also lower at 570.00 ms versus 720.00 ms (-20.8%), and mean P99 was 1276.67 ms versus 1380.00 ms (-7.5%). Despite those latency results, EC2 achieved the higher throughput, 43.72 requests/s compared with 40.00 requests/s (-8.5% for Serverless). The reliability gap was much larger than in W02: EC2 remained at 0.000% mean failures, while Serverless reached 4.086%, a +4.086 percentage-point difference. Estimated cost per 1,000 successful requests was \$0.000194 on EC2 and \$0.001786 on Serverless, a 9.21× ratio. W03 therefore shows why lower latency alone is not enough to declare an architecture better.

In other words, the architecture with the lower latency did not also deliver the best throughput, reliability, and cost. The metrics need to be considered together rather than reduced to a single score.

5.4 Sudden Burst

The sudden-burst workload produced the clearest formal responsiveness advantage for Serverless. Mean trial-level average latency was 152.77 ± 20.63 ms on Serverless compared with 225.00 ± 10.43 ms on EC2, a 32.1% reduction. Mean P95 fell from 400.00 ms to 300.00 ms (-25.0%), and mean P99 from 563.33 ms to 513.33 ms (-8.9%). Serverless also completed more work, averaging 59.26 requests/s versus 51.78 requests/s on EC2 (+14.5%). The trade-off was visible in reliability and cost: EC2 recorded a 0.000% mean failure rate, Serverless 0.434% (+0.434 percentage points), and the estimated cost per 1,000 successful requests was \$0.000154 for EC2 versus \$0.001463 for Serverless. The resulting cost ratio was 9.50×.

W04 therefore captures the main trade-off observed in this study: Serverless reacted more favorably to the abrupt burst on latency and throughput, while the tested EC2 setup remained cheaper and more reliable.

5.5 Idle-to-Burst

After the 300-second idle period, Serverless again showed a clear responsiveness advantage. Mean trial-level average latency was 168.64 ± 22.10 ms compared with 290.49 ± 142.04 ms on EC2 (-41.9%). Mean P95 was 340.00 ms versus 396.67 ms (-14.3%), and mean P99 was 676.67 ms versus 723.33 ms (-6.5%). Serverless throughput averaged 57.39 requests/s, compared with 44.04 requests/s on EC2 (+30.3%). EC2 recorded no request failures, while Serverless had a 0.916% mean failure rate (+0.916 percentage points). Cost was much closer than in the active 60-second workloads: \$0.001222 per 1,000 successful requests for EC2 and \$0.001297 for Serverless, only 1.06× higher for Serverless.

The narrowed cost gap is a direct consequence of the workload design. EC2 remained provisioned during the five-minute idle interval, while Lambda had no on-demand compute allocation during that idle period. This result should be interpreted as an idle-to-burst cost comparison. It is not evidence of a cold start because the experiment did not directly verify whether Lambda created a new execution environment after the idle interval.

5.6 Sustained Workload

The five-minute sustained workload produced a mixed result. Serverless had slightly lower mean trial-level average latency at 544.83 ± 250.63 ms, compared with 572.17 ± 449.46 ms on EC2 (-4.8%), and slightly higher achieved throughput at 21.84 versus 20.64 requests/s (+5.8%). Tail latency moved strongly in the other direction: mean P95 was 1566.67 ms on Serverless versus 996.67 ms on EC2 (+57.2%), and mean P99 was 4233.33 ms versus 2373.33 ms (+78.4%). Reliability also favored EC2, with mean failure rates of 0.110% and 1.354%, respectively, a +1.243 percentage-point difference for Serverless. Estimated cost per 1,000 successful requests was \$0.000461 on EC2 and \$0.001692 on Serverless, a 3.67× ratio.

W06 is a useful example of why an average can be misleading. Serverless remained competitive on mean latency and throughput, yet its P95/P99 latency and failure rate were less favorable over the longer run.

Table 4. Reliability Comparison

Workload	EC2 failure rate (%)	Serverless failure rate (%)	Difference (percentage points)	EC2 failed requests	Serverless failed requests	EC2 successful requests	Serverless successful requests
W01	0.000	0.000	+0.000 pp	0	0	1766	1771

Workload	EC2 failure rate (%)	Serverless failure rate (%)	Difference (percentage points)	EC2 failed requests	Serverless failed requests	EC2 successful requests	Serverless successful requests
W02	0.000	0.115	+0.115 pp	0	5	4679	4902
W03	0.000	4.086	+4.086 pp	0	278	7744	7034
W04	0.000	0.434	+0.434 pp	0	46	9226	10517
W05	0.000	0.916	+0.916 pp	0	90	7845	9970
W06	0.110	1.354	+1.243 pp	16	275	19598	21315

Note: Failure-rate difference is expressed in percentage points.

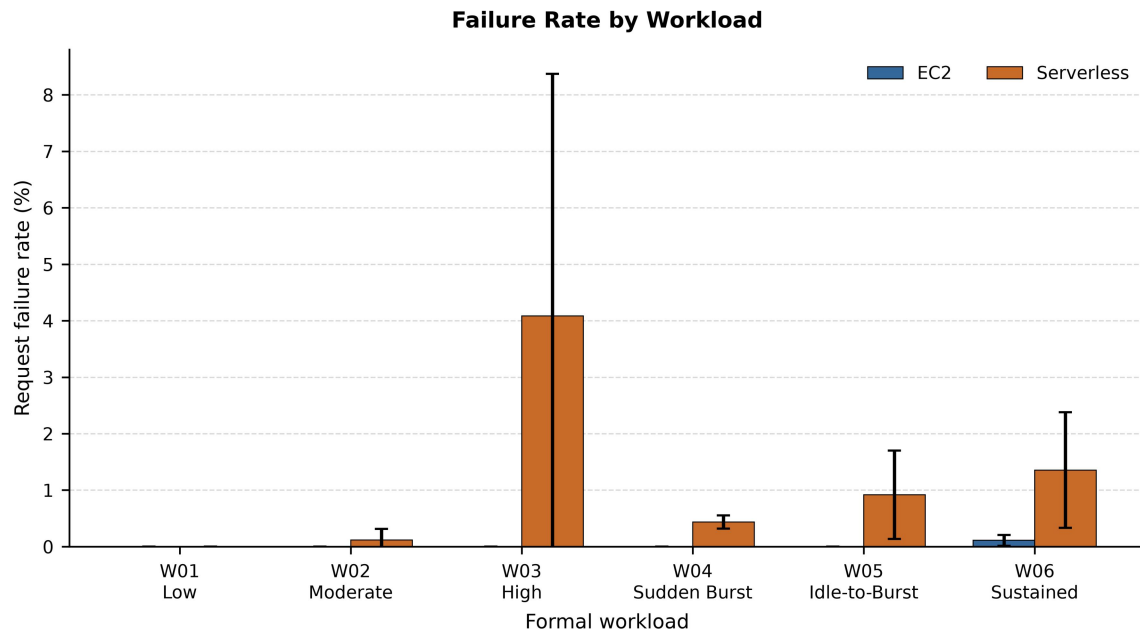


Figure 5. Formal request failure rate by workload.

5.7 Cost Comparison

Table 5. Estimated Cost Comparison

Workload	EC2 estimated cost / trial (USD)	Serverless estimated cost / trial (USD)	EC2 cost / 1,000 success (USD)	Serverless cost / 1,000 success (USD)	Serverless vs EC2 cost Δ	Serverless / EC2 cost ratio
W01	0.00047356	0.00087629	0.00080447	0.00148451	+84.5%	1.845
W02	0.00047356	0.00205757	0.00031057	0.00129150	+315.9%	4.159
W03	0.00047356	0.00352736	0.00019401	0.00178594	+820.5%	9.205
W04	0.00047356	0.00512335	0.00015402	0.00146294	+849.8%	9.498
W05	0.00284133	0.00427440	0.00122208	0.00129720	+6.1%	1.061
W06	0.00236778	0.01199757	0.00046144	0.00169234	+266.8%	3.668

Note: Costs use the regional AWS list-price methodology described in Section 3.9; Free Tier, credits, and discounts are excluded.

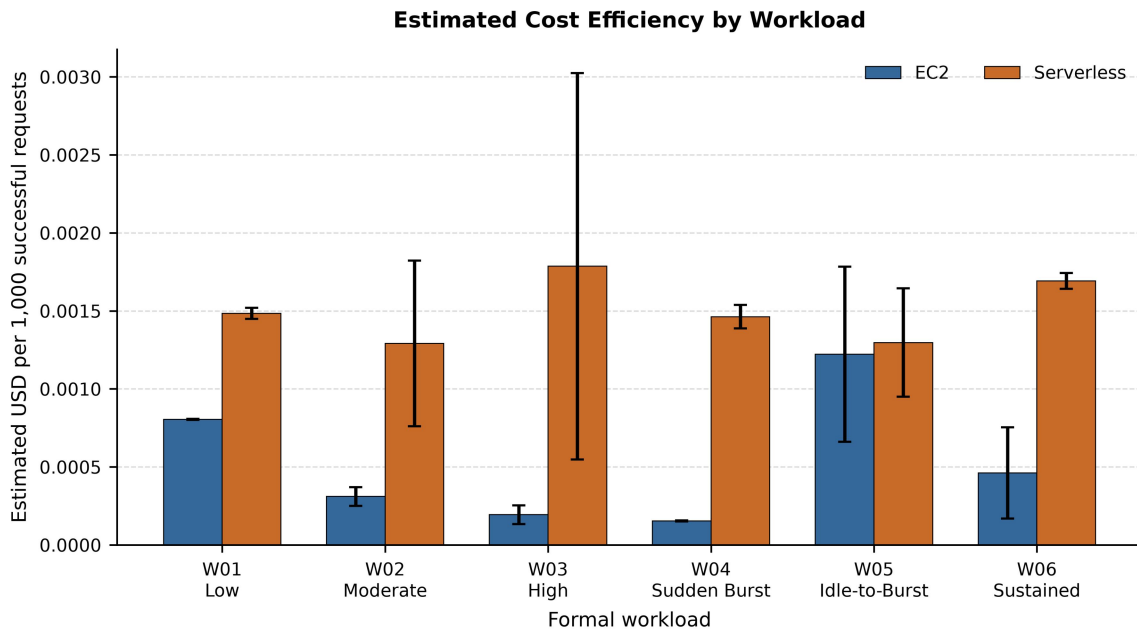


Figure 6. Estimated cost per 1,000 successful requests.

Across the six workloads, the tested EC2 configuration usually had the lower estimated cost per 1,000 successful requests. In the short, actively loaded tests, the continuously provisioned t3.small spread its allocation cost over a relatively large number of completed requests. The serverless path, by comparison, accumulated both Lambda request/duration charges and API Gateway request charges. W05 changed that balance because EC2 was charged through the 300-second idle period. Once that idle capacity was included, Serverless became much more cost-competitive, although its reliability result still differed from EC2.

Table 6. Three-Trial Summary and Variability

Workload	Architecture	n	Latency mean $\pm$ SD (ms)	Latency min-max (ms)	P95 mean $\pm$ SD (ms)	Throughput mean $\pm$ SD (req/s)	Throughput min-max (req/s)	Failure rate mean $\pm$ SD (%)	Cost/1K mean $\pm$ SD (USD)
W01	EC2	3	123.19 $\pm$ 2.99	119.79–125.37	196.67 $\pm$ 25.17	10.10 $\pm$ 0.06	10.06–10.17	0.000 $\pm$ 0.000	0.00080447 $\pm$ 0.00000393
W01	Serverless	3	126.04 $\pm$ 3.18	123.73–129.67	163.33 $\pm$ 25.17	10.05 $\pm$ 0.05	10.01–10.11	0.000 $\pm$ 0.000	0.00148451 $\pm$ 0.00003518
W02	EC2	3	200.09 $\pm$ 109.50	134.25–326.49	730.00 $\pm$ 840.77	26.70 $\pm$ 4.65	21.33–29.51	0.000 $\pm$ 0.000	0.00031057 $\pm$ 0.00005993
W02	Serverless	3	174.43 $\pm$ 53.99	138.62–236.53	426.67 $\pm$ 297.71	27.59 $\pm$ 2.70	24.50–29.54	0.115 $\pm$ 0.199	0.00129150 $\pm$ 0.00053157
W03	EC2	3	341.52 $\pm$ 220.69	208.22–596.26	720.00 $\pm$ 675.80	43.72 $\pm$ 11.76	30.15–51.03	0.000 $\pm$ 0.000	0.00019401 $\pm$ 0.00006020
W03	Serverless	3	325.82 $\pm$ 181.16	135.96–496.80	570.00 $\pm$ 547.45	40.00 $\pm$ 18.55	20.98–58.04	4.086 $\pm$ 4.286	0.00178594 $\pm$ 0.00123867
W04	EC2	3	225.00 $\pm$ 10.43	214.38–235.24	400.00 $\pm$ 55.68	51.78 $\pm$ 0.97	50.88–52.81	0.000 $\pm$ 0.000	0.00015402 $\pm$ 0.00000282
W04	Serverless	3	152.77 $\pm$ 20.63	134.47–175.13	300.00 $\pm$ 147.31	59.26 $\pm$ 2.58	56.57–61.70	0.434 $\pm$ 0.118	0.00146294 $\pm$ 0.00007451
W05	EC2	3	290.49 $\pm$ 142.04	205.96–454.48	396.67 $\pm$ 210.79	44.04 $\pm$ 15.98	25.58–53.58	0.000 $\pm$ 0.000	0.00122208 $\pm$ 0.00056053
W05	Serverless	3	168.64 $\pm$ 22.10	155.74–194.15	340.00 $\pm$ 121.24	57.39 $\pm$ 2.33	54.70–58.82	0.916 $\pm$ 0.782	0.00129720 $\pm$ 0.00034680
W06	EC2	3	572.17 $\pm$ 449.46	236.72–1082.85	996.67 $\pm$ 782.33	20.64 $\pm$ 13.59	6.24–33.24	0.110 $\pm$ 0.096	0.00046144 $\pm$ 0.00029170
W06	Serverless	3	544.83 $\pm$ 250.63	307.47–806.90	1566.67 $\pm$ 503.32	21.84 $\pm$ 8.49	12.97–29.89	1.354 $\pm$ 1.024	0.00169234 $\pm$ 0.00005061

Note: SD is sample standard deviation across $n=3$ trials. These descriptive statistics are not significance tests.

Several workload/architecture pairs show substantial variation across the three runs, which is why Table 6 is important to the interpretation. With $n=3$, the reported differences should be read as measurements from this controlled experiment rather than as precise population-level effects.

6. Discussion

6.1 Workload-Dependent Performance

The main finding is that the preferred architecture changed with workload shape. At W01, both systems handled the light workload with similar throughput, so cost became the clearest differentiator. At higher loads, Serverless often improved one or more latency measures, especially in the sudden-burst and idle-to-burst profiles. Those gains did not consistently extend to reliability or cost. The W03 and W06 results are particularly useful because they show that elasticity does not guarantee better throughput, tail latency, or failure behavior in every condition.

The sustained workload makes this point especially clear. Serverless was 4.8% lower on mean latency and 5.8% higher on throughput relative to EC2, yet its P95 latency was 57.2% higher and its mean failure rate was 1.354% compared with 0.110% on EC2. Looking only at the average would therefore give an incomplete account of the behavior observed in W06.

6.2 Resource and Scaling Behavior

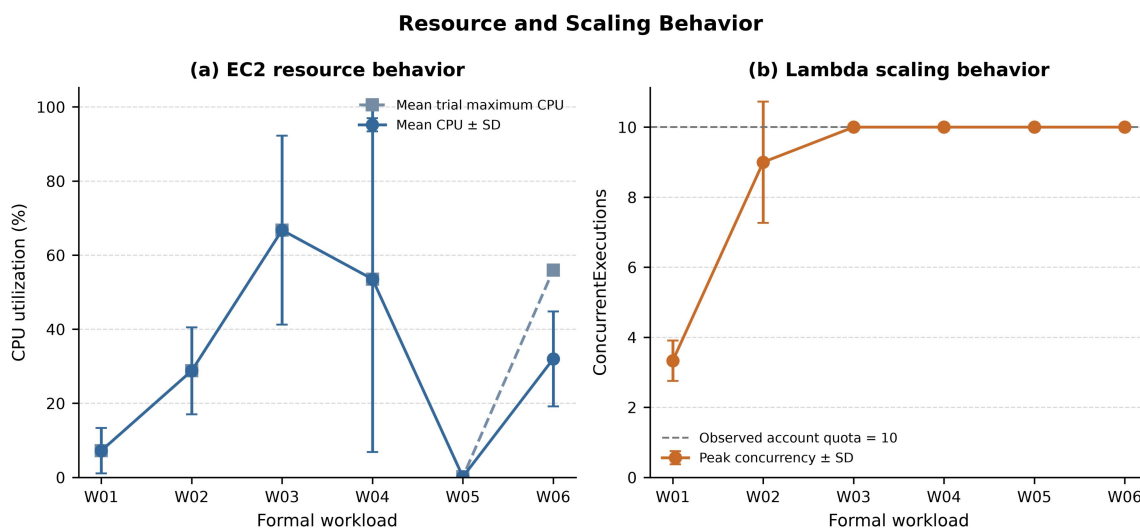


Figure 7. EC2 resource behavior and Lambda scaling behavior across formal workloads.

EC2 CPU utilization and Lambda concurrency describe different things and are shown separately in Figure 7. The EC2 panel indicates how the selected t3.small consumed its provisioned CPU capacity, while the Lambda panel shows the peak number of concurrent function executions. Peak Lambda concurrency reached the experimental quota value of 10 in W02, W03, W04, W05, and W06. The same five workloads also recorded nonzero Lambda throttles.

When a workload reached concurrency 10 and CloudWatch also recorded throttling, the observed degradation is consistent with exhaustion of the account's available concurrency. Failures that occurred without that combination are not automatically attributed to the quota. Keeping those cases separate is important because the account limit was an environmental constraint, not a property of Lambda as a service.

AWS documentation describes a default regional account concurrency limit of 1,000 concurrent executions, although quotas can vary by account and circumstances, and functions can be throttled when the available pool is exhausted [10]. The value of 10 observed in this experiment was therefore unusually restrictive. It materially limits how far the higher-concurrency serverless results can be generalized.

6.3 Reliability Trade-offs

EC2 was generally the more reliable architecture in the formal runs, particularly as the workload became heavier. Request failures were not removed as “bad runs”; when the instrumentation itself was valid, those failures remained part of the measured outcome. The runner distinguished application/HTTP failures from instrumentation problems so that nonzero failure rates were preserved rather than cleaned away.

For Serverless, the reliability result covers the complete HTTP path through API Gateway and Lambda, not only successful handler executions. API Gateway 5XX responses, Lambda throttles, and Lambda errors are therefore retained as separate fields in the dataset. This avoids treating every 5XX response as if it were necessarily caused by application code inside the Lambda function.

6.4 Cost Trade-offs

The cost comparison reflects the basic difference between provisioned capacity and usage-based execution. During short periods of active load, the low-priced t3.small produced a very small normalized cost because the instance was already available and was kept busy by the test. The serverless path accumulated API Gateway request charges together with Lambda request and duration charges. W05 changed the picture because the EC2 instance remained allocated throughout the intentional idle interval, while Lambda compute did not.

These values should not be read as a universal monthly cost forecast. Real applications differ in idle time, request rate, function duration, memory, data transfer, operational overhead, scaling policy, discounts, and reserved-capacity commitments. The result is narrower: under the fixed application and workload definitions used here, changing the idle/burst structure was enough to change the relative cost gap substantially.

6.5 Answers to the Research Questions

RQ1 — Performance: Neither architecture was best across all workloads and metrics. Serverless often reduced latency in the burst-oriented profiles and sometimes increased achieved throughput, while EC2 remained competitive or better in other tail-latency conditions. W06 is the clearest example of mean latency and tail latency pointing in different directions.

RQ2 — Reliability and scaling: EC2 was generally more reliable in the formal trials. Serverless behavior at higher concurrency was influenced by the account-specific Lambda concurrency quota, so failures in those workloads need to be interpreted alongside ConcurrentExecutions and Throttles rather than generalized to an unrestricted Lambda account.

RQ3 — Cost: The tested EC2 setup usually had the lower estimated cost per 1,000 successful requests during actively loaded periods. The gap narrowed sharply in W05 because the EC2 instance remained provisioned during the 300-second idle interval while Lambda on-demand compute did not.

RQ4 — Resource/account constraints: EC2 CPU utilization measures how much of a fixed provisioned resource was being consumed, whereas Lambda concurrency counts simultaneous function executions. They are complementary measurements, not quantities that can be compared on one numerical scale. The concurrency ceiling of 10 belongs to the experimental account and is treated as a threat to external validity rather than as a general Lambda limit.

7. Threats to Validity

7.1 Internal Validity

Cloud experiments inevitably contain background variability. Shared infrastructure, network conditions, provider scheduling, execution-environment reuse, and VM placement can influence individual runs. Repeating each condition three times and reporting sample standard deviations reduces the dependence on a single observation, but it does not remove this variability.

The EC2 system also used a T3 burstable instance. CPU-credit state can affect performance, and T3 Unlimited mode can allow the instance to run above its baseline and may introduce surplus-credit charges under prolonged utilization [14]. CPU utilization was monitored, but the results are not presented as representative of fixed-performance EC2 instance families.

CloudWatch provides infrastructure-level context rather than request-level causality in this experiment. Metrics were extracted as one-minute datapoints for the requested UTC windows, so a CloudWatch average cannot be matched precisely to a particular Locust request at millisecond resolution.

All formal trials came from a single load-generator environment. A pre-formal check did not show obvious client CPU saturation, but the network path and client-side scheduling can still contribute to run-to-run variation.

7.2 External Validity

The experiment covers one AWS Region (ap-south-1), one EC2 type (t3.small), one Lambda memory setting (1024 MB), one Lambda architecture (x86_64), one API type (HTTP API), and one deterministic CPU-bound application. I/O-bound services, databases, larger payloads, other EC2 families, Graviton/ARM, different Lambda memory allocations, provisioned or reserved concurrency, and other Regions may produce different results.

The absolute concurrency in this study is modest compared with many production systems. Large applications may also use caches, CDNs, queues, asynchronous processing, Auto Scaling groups, load balancers, reserved capacity, or multi-region routing. The results should therefore be read as a controlled case study rather than as a benchmark of the maximum capacity of either AWS service.

7.3 Construct Validity

Locust was configured as a closed-loop user model. Because faster responses allow the same virtual users to issue their next requests sooner, requests_per_second represents achieved throughput under the common concurrency pattern; it is not a fixed offered arrival rate.

W05 includes a 300-second idle interval, but AWS does not guarantee that a Lambda execution environment will be discarded after any fixed period. Execution environments may be retained and reused [11]. The workload is therefore described as idle-to-burst and not as proof of a cold start.

The cost model has a similar limitation. Lambda GB-second cost is estimated from aggregate invocation count, average CloudWatch Duration, configured memory, and the regional list price. Exact invoice-level billed duration can differ, so the paper reports estimated rather than actual per-trial cost.

7.4 AWS Account Concurrency Limitation

The most important serverless-specific constraint in the experiment was the observed regional Lambda concurrency quota of 10. During pilot calibration, the original 50-user sudden burst reached that ceiling and produced throttling together with HTTP 503 responses. The formal burst was reduced, but several higher-load formal conditions could still approach the same account limit.

This should not be read as evidence that “AWS Lambda scales to only 10 concurrent requests.” AWS documents a much larger default regional concurrency pool and supports quota management; functions are throttled when the available concurrency is exhausted [10]. The high-concurrency serverless results in this paper therefore describe Lambda under this particular account constraint, not the unconstrained scaling ceiling of the service.

8. Reproducibility and Data Availability

The study was built as a reproducible infrastructure-and-analysis workflow. Terraform provisioned the EC2 and Serverless environments, and a shared Locust workload file generated requests against /compute. Each trial recorded its architecture, workload, trial number, UTC timing, application commit, and Terraform commit. Raw Locust outputs, CloudWatch query results, processed datasets, cost calculations, statistical summaries, tables, and figures were generated through scripts rather than by manually copying values into the paper.

The processed dataset and non-sensitive reproducibility materials are included with the preprint as supplementary material. The complete research repository remains private because it also contains account-specific logs and security-sensitive configuration. Additional non-sensitive supporting material can be shared by the corresponding author on reasonable request.

The main dataset is data/final/experiments.csv, containing 36 rows and 44 columns. Supporting artifacts include workload/architecture summaries, architecture comparisons, the pricing snapshot, cost breakdowns, generated tables and figures, and SHA-256 manifests. Pilot calibration material is kept separate and is not included in the formal result graphs.

9. Conclusion

This study compared a fixed-capacity EC2 web deployment with an API Gateway + AWS Lambda serverless deployment across six predefined workload patterns while keeping the application computation identical. Across 36 formal trials, the outcome depended on workload shape. Serverless often responded better to sudden or idle-to-burst traffic, and in W05 its relative cost improved markedly because compute was not continuously allocated during the idle interval. The tested EC2 setup, however, was generally less expensive per 1,000 successful requests and more reliable in the measured conditions.

The experiment also shows why a single metric is not enough for this type of comparison. Average latency, P95/P99 tail latency, achieved throughput, failure rate, resource behavior, and estimated cost did not always favor the same architecture within a workload. W06 is a good example: competitive mean behavior coexisted with less favorable tail latency and reliability on the serverless path.

The serverless results also need to be read in light of the account-specific Lambda concurrency quota of 10. Measurements taken near that ceiling are valid observations of the tested environment, but they do not describe the normal scaling limit of AWS Lambda. Future work should repeat the same workload matrix with a higher concurrency quota, other EC2 families, additional Lambda memory and CPU architectures, open-loop arrival processes, and more AWS Regions. That would show whether the workload-dependent trade-offs observed here remain consistent under broader capacity and traffic conditions.

References

- [1] J. M. Hellerstein, J. Faleiro, J. E. Gonzalez, J. Schleier-Smith, V. Sreekanti, A. Tumanov, and C. Wu, “Serverless Computing: One Step Forward, Two Steps Back,” in *9th Biennial Conference on Innovative Data Systems Research (CIDR)*, 2019.
- [2] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, “Peeking Behind the Curtains of Serverless Platforms,” in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pp. 133–146, 2018.
- [3] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, “Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider,” in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pp. 205–218, 2020.

- [4] A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, “Firecracker: Lightweight Virtualization for Serverless Applications,” in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pp. 419–434, 2020.
- [5] I. E. Akkus, R. Chen, I. Rımac, M. Stein, K. Satzke, A. Beck, P. Aditya, and V. Hilt, “SAND: Towards High-Performance Serverless Computing,” in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pp. 923–935, 2018.
- [6] S. Boucher, A. Kalia, D. G. Andersen, and M. Kaminsky, “Putting the ‘Micro’ Back in Microservice,” in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pp. 645–650, 2018.
- [7] S. Shillaker and P. Pietzuch, “Faasm: Lightweight Isolation for Efficient Stateful Serverless Computing,” in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pp. 419–433, 2020.
- [8] C. Lin, N. Mahmoudi, S. Fan, and H. Khazaei, “Fine-Grained Performance and Cost Modeling and Optimization for FaaS Applications,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 1, pp. 180–194, 2023, doi: 10.1109/TPDS.2022.3214783.
- [9] J. Dean and L. A. Barroso, “The Tail at Scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013, doi: 10.1145/2408776.2408794.
- [10] Amazon Web Services, “Understanding Lambda function scaling,” *AWS Lambda Developer Guide*, accessed Aug. 17, 2026.
- [11] Amazon Web Services, “Understanding the Lambda execution environment lifecycle,” *AWS Lambda Developer Guide*, accessed Aug. 17, 2026.
- [12] Amazon Web Services, “AWS Lambda Pricing,” accessed Aug. 17, 2026.
- [13] Amazon Web Services, “Purchasing On-Demand Instances for Amazon EC2,” *Amazon EC2 User Guide*, accessed Aug. 17, 2026.
- [14] Amazon Web Services, “Key concepts for burstable performance instances,” *Amazon EC2 User Guide*, accessed Aug. 17, 2026.
- [15] Amazon Web Services, “Amazon API Gateway Pricing,” accessed Aug. 17, 2026.